Moayd Shahat

Software Engineering Student · KFUPM

Assignment 4 Final Portfolio
Professional Demonstration

Static SPA	API	Sections	Docs
Vanilla JS	**GitHub REST**	**6 Core**	**6 Files**

01 · Introduction

Project overview and personal motivation

Project Overview

- A static Single-Page Application built with HTML, CSS, and vanilla JavaScript.
- Extends Assignment 3 into a polished final portfolio ready for public deployment.
- Deployed on GitHub Pages — accessible at any time without a server.
- Integrates the public GitHub API to show live repository data.
- Validated contact form with real accessibility support (ARIA).
- Complete documentation: README, Technical Docs, User Guide, AI Report.

Personal Motivation

🎯 **Real-World Skills**
Apply course concepts to a deployable project, not just an exercise.

🤖 **AI-Assisted Workflow**
Explore responsible use of AI tools to write, review, and document code.

📱 **Responsive Design**
Build a portfolio that works on every screen size without a framework.

📄 **Documentation Culture**
Practice writing clear technical docs — a skill every engineer needs.

🔗 **API Integration**
Gain hands-on experience consuming REST APIs with error handling.

02 · Technical Demonstration

Live demonstration of key features

🌓 Dark / Light Theme

Toggle persists across page reloads via localStorage. Body class switches the entire CSS token system.

🔍 Project Filtering & Search

Full-text search + category, complexity, and sort filters. Live counters update with each change.

🔗 GitHub API Integration

Fetches 6 latest repos with AbortController timeout. Falls back to local cards on network failure.

📋 Contact Form Validation

Name, email pattern, inquiry type, and 20-char message all validated with ARIA error states.

💾 Persistent Visitor State

Visitor name and availability preference saved in localStorage and restored on every visit.

⏱ Session Timer

Live timer tracks session duration each second. Resets on reload — no server needed.

02 · Technical Architecture Overview

How the application is structured and how data flows

Presentation Layer

· index.html (semantic HTML5)

· css/styles.css (CSS Grid, Custom Properties, Dark Mode)

Application Logic

· js/script.js — state object, elements map, storageKeys

· Rendering · Event handling · Filtering · Sorting · Validation

External Services

· GitHub Public API → fetch() + AbortController (6s timeout)

· localStorage → theme, visitor name, availability status

Key Design Decisions

- Zero frameworks — fast, no build step
- One state object — single source of truth
- DOM methods only — no innerHTML injection
- storageKeys map — no magic strings
- AbortController — resilient API calls
- ARIA attributes — accessible by default
- CSS Custom Properties — instant theming
- Fallback repo cards — graceful failure

Page Load Flow: Read localStorage → Attach event listeners → Render UI → Fetch GitHub API (async) → Timer starts

02 · AI Integration Highlights

How AI tools were used responsibly in the development workflow

🤖 Code Generation

Drafted JavaScript state management, filter logic, and card rendering. Each block reviewed, tested, and manually adjusted before use.

🤖 Documentation

Outlined README, technical docs, and user guide structure. All content verified for accuracy against actual implementation.

🤖 CSS Assistance

Generated dark mode token system, responsive breakpoints, and focus styles. Visual output verified at every screen width.

🤖 Accessibility Review

Identified missing ARIA attributes and skip-link patterns. Applied only after validating against WCAG guidelines.

Responsible Use Principles Applied:

- ✓ Verify every suggestion
- ✓ Never blindly copy-paste
- ✓ Test in-browser
- ✓ Document AI usage transparently

03 · Technical Deep Dive

Challenges solved, limitations, future work, and lessons learned

✅ Resolved Challenges

- API timeout — AbortController cancels after 6 s
- State drift — single state object + re-render on change
- Theme flash — CSS tokens applied before render
- Unsafe HTML — DOM methods replace innerHTML
- Filter coupling — getFilteredProjects() is the only source
- ARIA gaps — aria-invalid, aria-live added to all forms

🚀 Planned Future Work

· Formspree/EmailJS for contact form submission

· Service Worker for offline-first caching

· Paginated project grid with lazy loading

💡 Lessons Learned

Design for failure first

API errors are common — fallback UI saves the demo.

State before DOM

One state object prevents bugs that come from reading the DOM as truth.

Docs are part of the code

Writing the README revealed gaps that testing alone missed.

AI needs review

Generated code was a starting point, not a final answer.

04 · Conclusion

Project Outcomes & Achievements

6	6	100%	4+
Core Features	**Documentation files**	**Vanilla JS**	**Responsive layouts**
Implemented & tested	README to AI Report	Zero frameworks	Mobile to desktop

🔭 Future Improvements & Extensions

→ Add Formspree backend for real contact form submissions
→ Add GitHub Actions for automated deployment and testing
→ Implement Service Worker for offline-first PWA support
→ Integrate a CMS headless layer for content editing without code
→ Expand project section with pagination and tag cloud

🌐 https://swe-moayd.github.io/202277460-moayd-shahat-assignment4/